

ROS 1일차 과제3 보고서, 김도훈, 2026406041

<목차>

1. 과제3 설명
2. 과제3 구현
3. 실행 결과

1. 과제3 설명

과제3은 터틀심과 터틀심의 토픽을 사용해서 프로그램 실행 시 거북이가 색깔과 굵기가 다른 도형을 그리고 도형을 다 그리고 난 후에는 키 입력을 받아 입력에 해당하는 도형을 그리도록 프로그램을 구현하는 것을 목표로 하는 과제이다.

2. 과제3 구현

프로그램을 실행하면 먼저 자동으로 다음 순서대로 도형을 그리도록 구성하였다.
빨간색 사각형 -> 노란색 삼각형 -> 파란색 원 -> W/A/S/D 입력 대기

거북이 이동 구현

거북이의 이동을 위해 `geometry_msgs/msg/Twist` 메시지를 사용하였다.

```
cmd_pub_ = create_publisher<geometry_msgs::msg::Twist>("/turtle1/cmd_vel", 10);
```

`linear.x` 값을 이용하여 직진 속도를 설정하고 `angular.z` 값을 이용하여 회전하도록 구현하였다.

```
msg.linear.x = 1.5;  
msg.angular.z = 0.0;  
cmd_pub_->publish(msg);
```

직진할 때는 `linear.x`에 값을 주고 방향을 변경할 때는 `angular.z` 값을 이용하였다.

사각형과 삼각형 구현

사각형과 삼각형은 직진과 회전을 반복하는 방식으로 구현하였다.

사각형은 총 4개의 변을 그리며 한 변을 그린 후 90도씩 회전하도록 설정하였다.

```
target_sides_ = 4;  
turn_angle_ = PI / 2.0;  
=
```

삼각형은 3개의 변을 그리고 한 변이 끝날 때마다 120도씩 회전하도록 구현하였다.

```
target_sides_ = 3;
turn_angle_ = 2.0 * PI / 3.0;
```

따라서 변의 개수와 회전각을 변경하는 방법으로 여러 종류의 다각형을 구현할 수 있도록 하였다.

원 그리기 구현

원은 거북이가 직진과 회전을 동시에 하도록 하여 구현하였다.

```
msg.linear.x = 1.5;
msg.angular.z = 1.0;
```

linear.x와 angular.z에 동시에 값을 주면 거북이가 앞으로 이동하면서 계속 회전하게 되고 이를 이용하여 원을 그릴 수 있다.

Pose를 이용한 이동 보정

처음에는 일정 시간 동안 직진하거나 회전하는 방식으로 구현하였으나 Timer의 실행 시간에 따라 회전각에 오차가 발생하여 사각형이 정확하게 그려지지 않는 문제가 있었다.

이를 해결하기 위해 /turtle1/pose Topic을 구독하여 거북이의 실제 위치와 방향을 확인하도록 수정하였다.

```
pose_sub_ = create_subscription<turtlesim::msg::Pose>( "/turtle1/pose", 10, std::bind(
&TurtleShape::pose_callback, this, std::placeholders::_1));
```

수신한 x, y, theta 값을 이용해 실제 이동 거리와 회전각을 계산하였다. 따라서 단순히 시간만 이용하는 방식보다 정확하게 도형을 그릴 수 있도록 하였다.

도형 색상과 굵기 변경

도형마다 다른 색상과 굵기를 적용하기 위해 turtlesim의 /turtle1/set_pen Service를 사용하였다.

```
request->r = r;
request->g = g;
request->b = b;
request->width = width;
request->off = 0;
```

각 도형은 다음과 같이 설정하였다.

사각형 : 빨간색 / 굵기 3

삼각형 : 노란색 / 굵기 5

원 : 파란색 / 굵기 7

도형을 그리기 전에 set_pen Service를 호출하여 색상과 굵기를 변경하도록 하였다.

도형 위치 변경 여러 도형이 서로 겹치지 않도록 /turtle1/teleport_absolute Service를 이용하였다.

```
request->x = x;
```

```
request->y = y;
```

```
request->theta = theta;
```

거북이를 새로운 위치로 이동할 때는 펜을 잠시 끈 후 이동시키고, 이동이 끝나면 다시 펜을 켜도록 하여 이동 경로가 화면에 그려지지 않도록 하였다.

W/A/S/D 입력을 통한 도형 선택

기본 도형 3개를 모두 그린 이후에는 사용자에게 키보드 입력을 받아 추가 도형을 그릴 수 있도록 구현하였다. W : 사각형, A : 삼각형, S : 원, D : 오각형 입력된 문자에 따라 도형의 변 개수, 회전각, 색상 등을 설정한 뒤 해당 도형을 그리도록 하였다. 예를 들어 사각형은 다음과 같이 선택한다.

```
if (key == 'W')
```

```
{    target_sides_ = 4;
```

```
    turn_angle_ = PI / 2.0;
```

```
}
```

도형이 완성되면 다시 W/A/S/D 입력을 받을 수 있도록 하여 프로그램을 종료하지 않고 반복적으로 원하는 도형을 그릴 수 있도록 구현하였다.